

Consolidação Prática da Organização Profissional de Projetos Python com venv e requirements.txt

Consolidação Estrutural do Projeto Python

Um projeto Python profissional não é apenas um arquivo que executa código. Ele envolve uma estrutura organizada que permite separar responsabilidades, facilitar manutenção, tornar o sistema compreensível e possibilitar que o ambiente de execução seja reconstruído em outras máquinas.

A consolidação prática desta estrutura reúne conceitos já trabalhados anteriormente, integrando:

- organização modular do código
- separação de responsabilidades entre componentes
- utilização de pacotes Python
- isolamento do ambiente com venv
- registro de dependências com requirements.txt
- reconstrução do ambiente em outra máquina

Essa integração permite compreender o projeto Python como uma estrutura de sistema, e não apenas como um script isolado.

1. Retomada Estrutural da Organização de Projetos

Scripts simples costumam concentrar toda a lógica em um único arquivo. Essa abordagem funciona para experimentos ou programas muito pequenos, porém rapidamente se torna difícil de manter quando o código cresce.

Projetos estruturados adotam uma divisão clara de responsabilidades entre arquivos e pastas. Cada parte do sistema assume um papel específico dentro da arquitetura.

Essa separação melhora diversos aspectos do desenvolvimento:

- facilita a leitura do código
- torna a manutenção mais previsível
- permite crescimento organizado do sistema
- possibilita trabalho colaborativo entre desenvolvedores

Em ambientes profissionais, cada componente do projeto possui uma função clara dentro da arquitetura do sistema.

2. Estrutura do Projeto

A estrutura construída possui a seguinte organização:

```
projeto_cadastro_clientes/
├── main.py
├── validacoes/
│   ├── __init__.py
│   └── regras_validacao.py
├── servicos/
│   ├── __init__.py
│   └── cadastro.py
├── dados/
│   ├── __init__.py
│   └── armazenamento.py
└── interface/
    ├── __init__.py
    └── exibicao.py
```

Cada diretório representa uma área específica do sistema.

- **validacoes** → responsável por verificar se os dados são válidos
- **servicos** → responsável por aplicar as regras principais do sistema
- **dados** → responsável por simular armazenamento de informações
- **interface** → responsável por exibir informações ao usuário
- **main.py** → responsável por coordenar o fluxo geral do programa

O arquivo **__init__.py** formaliza cada pasta como um pacote Python, permitindo que seus módulos sejam importados de forma hierárquica dentro do projeto.

Os arquivos .py dentro de cada pacote guardam partes específicas da lógica do sistema.

3. Criação do Ambiente Virtual

No terminal do VS Code, a criação do ambiente virtual é feita com o comando:

```
python -m venv venv
```

Após a criação, o ambiente é ativado no Windows com:

```
venv\Scripts\activate
```

O ambiente virtual cria uma instalação isolada do Python associada apenas ao projeto atual.

Esse isolamento evita conflitos entre bibliotecas e versões utilizadas em diferentes projetos. Cada projeto passa a possuir seu próprio conjunto de dependências, mantendo a instalação global do Python intacta.

Como o conceito de ambiente virtual já foi estudado anteriormente, neste momento a retomada ocorre apenas como reforço funcional dentro da montagem do projeto.

4. Módulo de Validação

Arquivo:

validacoes/regras_validacao.py

Código:

```
# Função responsável por validar se o nome possui conteúdo suficiente
def validar_nome(nome):
    # Remove espaços extras no início e no fim do texto
    nome = nome.strip()

    # Verifica se o nome possui pelo menos 3 caracteres
    if len(nome) < 3:
        return False

    # Retorna verdadeiro quando o nome é considerado válido
    return True

# Função responsável por validar a idade informada
def validar_idade(idade):
    # Verifica se a idade está fora do intervalo aceitável
    if idade < 0 or idade > 120:
        return False

    # Retorna verdadeiro quando a idade é considerada válida
    return True
```

Esse módulo possui apenas a responsabilidade de verificar se as informações fornecidas são válidas.

Ele não executa cadastro, não imprime mensagens e não realiza armazenamento de dados. Sua função é apenas decidir se os valores recebidos atendem às regras definidas.

Essa separação caracteriza a ideia de responsabilidade específica, onde cada módulo do sistema executa apenas um tipo de tarefa.

5. Módulo de Armazenamento

Arquivo:

dados/armazenamento.py

Código:

```
# Lista que simula o armazenamento dos clientes cadastrados
clientes = [ ]

# Função responsável por salvar um cliente na lista
```

```
def salvar_cliente(nome, idade):  
    # Cria um dicionário com os dados do cliente  
    cliente = {  
        "nome": nome,  
        "idade": idade  
    }  
  
    # Adiciona o cliente na lista de armazenamento  
    clientes.append(cliente)
```

Nesse estágio do projeto ainda não existe integração com banco de dados.

O armazenamento é simulado utilizando uma lista em memória. Cada cliente é representado por um dicionário contendo os dados cadastrados.

Mesmo sendo uma simulação simples, a separação do armazenamento em um módulo próprio mantém a arquitetura organizada e evita misturar persistência de dados com validação ou interface.

6. Módulo de Serviços

Arquivo:

servicos/cadastro.py

Código:

```
# Importa as funções de validação  
from validacoes.regras_validacao import validar_nome, validar_idade  
  
# Importa a função de salvamento  
from dados.armazenamento import salvar_cliente  
  
# Função responsável por aplicar a lógica principal do cadastro  
def cadastrar_cliente(nome, idade):  
    # Verifica se o nome informado é válido  
    if not validar_nome(nome):  
        return "Nome inválido."  
  
    # Verifica se a idade informada é válida  
    if not validar_idade(idade):  
        return "Idade inválida."  
  
    # Salva o cliente quando os dados passam pelas validações  
    salvar_cliente(nome, idade)  
  
    # Retorna mensagem de sucesso  
    return "Cliente cadastrado com sucesso."
```

Esse módulo representa o centro da lógica de negócio do sistema.

Ele recebe os dados, executa as validações necessárias e, quando as informações são consideradas válidas, realiza o armazenamento.

O módulo de serviços não solicita entrada de dados e não exibe mensagens. Ele apenas executa a lógica do sistema e retorna o resultado da operação.

7. Módulo de Interface

Arquivo:

interface/exibicao.py

Código:

```
# Função responsável por exibir mensagens ao usuário
def mostrar_mensagem(mensagem):
    # Exibe uma linha visual superior
    print("\n=====")

    # Exibe a mensagem recebida
    print(mensagem)

    # Exibe uma linha visual inferior
    print("=====\n")
```

A camada de interface é responsável apenas por apresentar informações na tela.

Ela não valida dados, não executa regras de negócio e não realiza armazenamento. Essa separação ajuda a manter o sistema organizado e facilita alterações futuras na forma de apresentação das mensagens.

8. Programa Principal

Arquivo:

main.py

Código:

```
# Importa a função principal de cadastro
from servicos.cadastro import cadastrar_cliente

# Importa a função responsável por exibir mensagens
from interface.exibicao import mostrar_mensagem

# Função principal do sistema
def main():
    # Solicita o nome do cliente
    nome = input("Digite o nome do cliente: ")

    # Solicita a idade do cliente e converte para inteiro
    idade = int(input("Digite a idade do cliente: "))
```

```
# Executa a lógica de cadastro
resultado = cadastrar_cliente(nome, idade)

# Exibe o resultado na tela
mostrar_mensagem(resultado)

# Garante que o arquivo seja executado como ponto de entrada principal
if __name__ == "__main__":
    main()
```

O arquivo main.py atua como ponto de entrada do sistema.

Ele conecta os diferentes módulos do projeto, coordenando o fluxo geral do programa:

- coleta dados de entrada
- chama a lógica de serviços
- apresenta o resultado

O objetivo é centralizar o fluxo do sistema sem concentrar todas as regras no mesmo arquivo.

9. Teste Inicial do Sistema

A execução do sistema é realizada no terminal:

python main.py

Alguns cenários de teste simples ajudam a verificar o funcionamento da estrutura:

- nome válido e idade válida
- nome muito curto
- idade negativa
- idade acima de 120

Esses testes permitem observar que cada parte do sistema atua em conjunto dentro da arquitetura modular.

10. Criação do Arquivo requirements.txt

Mesmo em projetos que utilizam apenas recursos internos da linguagem, o procedimento profissional de registro de dependências deve ser conhecido.

O arquivo é criado no terminal com o comando:

pip freeze > requirements.txt

O comando pip freeze lista os pacotes instalados no ambiente virtual e registra essas dependências no arquivo requirements.txt.

Esse registro permite reconstruir o ambiente do projeto em outra máquina de forma consistente.

11. Estrutura Final do Projeto

Após a montagem completa, o projeto apresenta a seguinte estrutura:

```
projeto_cadastro_clientes/
├── main.py
├── requirements.txt
├── validacoes/
│   ├── __init__.py
│   └── regras_validacao.py
├── servicos/
│   ├── __init__.py
│   └── cadastro.py
├── dados/
│   ├── __init__.py
│   └── armazenamento.py
├── interface/
│   ├── __init__.py
│   └── exibicao.py
└── venv/
```

Cada componente possui um papel específico dentro da estrutura.

- venv representa o ambiente virtual da máquina atual
- requirements.txt registra as dependências do projeto
- os arquivos .py representam o código do sistema

Essa organização torna o projeto portátil e estruturado.

12. Desativação do Ambiente Virtual

Para sair do ambiente virtual, utiliza-se o comando:

deactivate

Após a desativação, o terminal volta a utilizar o Python global instalado na máquina.

13. Simulação de Transporte do Projeto

Uma prática comum no desenvolvimento de software é transportar o projeto entre máquinas.

Esse cenário pode ocorrer ao:

- levar o projeto para outro computador
- compartilhar o código com outra pessoa
- mover o sistema entre ambientes de desenvolvimento

Nesse fluxo, o projeto pode ser copiado para um pendrive, removido da máquina atual e posteriormente restaurado em outro ambiente.

O que realmente precisa ser transportado é:

- o código do projeto
- a estrutura de pastas
- o arquivo requirements.txt

O ambiente virtual pode ser recriado posteriormente.

14. Recriação do Ambiente Virtual

Após copiar novamente o projeto para a máquina, o ambiente virtual pode ser recriado com:

```
python -m venv venv
```

Em seguida, o ambiente é ativado:

```
venv\Scripts\activate
```

Esse procedimento recria o ambiente isolado do projeto.

15. Reinstalação das Dependências

Para reinstalar as dependências registradas no projeto, utiliza-se:

```
pip install -r requirements.txt
```

O comando faz com que o pip leia o arquivo e instale automaticamente os pacotes listados nele.

Isso garante que o ambiente reconstruído possua as mesmas dependências do projeto original.

16. Teste Final do Projeto Reconstruído

Após recriar o ambiente e instalar as dependências, o sistema pode ser executado novamente:

```
python main.py
```

Se todos os passos foram realizados corretamente, o projeto volta a funcionar normalmente.

Isso demonstra que a estrutura do sistema permanece íntegra mesmo após ser transportada e reconstruída em outro ambiente.

17. Consolidação Final

Um projeto Python organizado profissionalmente reúne diversos elementos estruturais:

- código modularizado
- pacotes organizados por responsabilidade
- separação clara entre validação, lógica, armazenamento e interface
- ambiente virtual isolado
- dependências registradas
- possibilidade de reconstrução do ambiente em outra máquina

Essa combinação transforma um conjunto de arquivos em um projeto estruturado, preparado para crescimento, manutenção e compartilhamento em diferentes ambientes de desenvolvimento.